

Parallelism in the commutation fold

From the algebra of series commutation to the execution schedule of a log-signature computation

signature-rs / lie-rs — 2026-09-02. Audience: readers comfortable with Lyndon bases, Lie series commutation, and log signatures; no familiarity with this codebase assumed. Concepts and methods only.

The three scales of the computation

A log signature of a path is built by folding the displacements of consecutive path points into a running Lie series, one Baker–Campbell–Hausdorff (BCH) concatenation at a time. The whole computation nests three scales, and the honest first answer to “where can threads go?” is different at each scale:

The path
(scale 1)

An associative reduction of the per-segment displacements by BCH concatenation: $S = (((d_1 \oplus d_2) \oplus d_3) \oplus \dots)$. Computed today as a **strictly sequential chain** — fold $k + 1$ ’s input is fold k ’s output.

serial between folds; associativity is the only escape hatch (§8)

The fold
(scale 2)

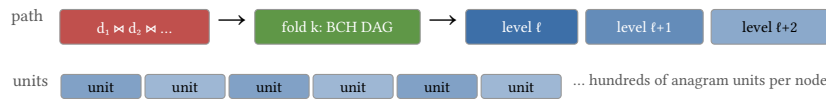
One concatenation $A \oplus B$ evaluates a fixed **directed acyclic graph** of Lie bracket nodes, compiled once from the BCH series truncated at the working degree. Nodes on one dependency level are independent; levels are dependency-ordered.

parallel within each level; barriers between levels (§5)

The commutation
(scale 3)

Each node evaluates one series commutation $[X, Y]$ over the Lyndon basis: thousands of independent pair contributions, partitioned into **anagram units** that never share a target word.

embarrassingly parallel inside a unit-partition (§2–4)



Nesting: a serial chain of folds; each fold a level-ordered DAG of commutations; each commutation a bag of anagram units. Parallelism exists at the two inner scales; the outermost scale is a data dependency, not a scheduling choice.

The algebra: what one commutation computes

Brackets over the Lyndon basis

A Lie series $A = \sum_u a_u w_u$ is a formal sum over the Lyndon words — the canonical basis of the free Lie algebra on d letters. The commutator of two series distributes over pairs of basis words:

$$[A, B] = \sum_{u < v} (a_u b_v - a_v b_u) [u, v]$$

The sum runs over **canonical pairs** $u < v$ of Lyndon words (antisymmetry halves the work immediately). Each bracket $[u, v] = uv - vu$ re-expands in the Lyndon basis; the expansion has two rigid structural symmetries, and they are the entire foundation of the parallel layout:

Degree additivity

$|u| + |v| = t$: every word in the expansion of $[u, v]$ has degree t . Truncating the series at degree m discards all pairs with $|u| + |v| > m$ up front.

Content homogeneity

$[u, v]$ expands only over words with the same letter multiset as uv . Writing $\gamma(w)$ for a word’s content vector in $\mathbb{N}^d$: a pair with $\gamma(u) + \gamma(v) = \gamma$ can only ever touch basis words of content γ .

A **feasible decomposition** of a basis word w is a canonical pair (u, v) such that w appears (with nonzero integer coefficient) in the expansion of $[u, v]$, together with that coefficient $c_{u,v}^w \in \mathbb{Q}$. The coefficient of w in $[A, B]$ is then one scalar fan-in over all its feasible decompositions:

$$[A, B]_w = \sum_{(u,v) \rightarrow w} c_{u,v}^w \cdot \underbrace{(a_u b_v - a_v b_u)}_{\text{term — 1-2 multiplies}}$$

Each feasible pair contributes its bracket-coefficient times the antisymmetric product of the operand values. All $c_{u,v}^w$ are exact small rationals — precomputable once per basis, before any data arrives.



One word w ’s coefficient = a short inner product: its (pair, rational) decomposition row dotted with the antisymmetric pair-terms. Both the pair table and the rational rows are **structure constants**: they depend only on the basis, never on the data.

The precomputed structure-constant table

Because every ingredient except the a_u and b_v is data-independent, the entire commutation is driven by a single precomputed table:

Entries

All feasible canonical pairs (i, j) , packed as tiny fixed-size records, **degree-blocked**: sorted by (target degree t , then $p = |u|$, $q = t - p$). Degree blocking is what lets a truncated fold skip whole (p, q) regions of the table without testing them.

Decomposition rows

Per entry, the run of (target word, coefficient) pairs it feeds — the words whose expansion contains $[u, v]$. Most rows have length 1–3: almost every pair feeds its own distinguished word (the word uv itself when it is Lyndon) and almost nothing else.

Anagram-unit bounds

Markers (§3) delimiting the groups of entries that can write to the **same** set of target words — the unit is the conflict boundary for parallel writes.

Anagram classes and units: the partition that makes parallelism safe

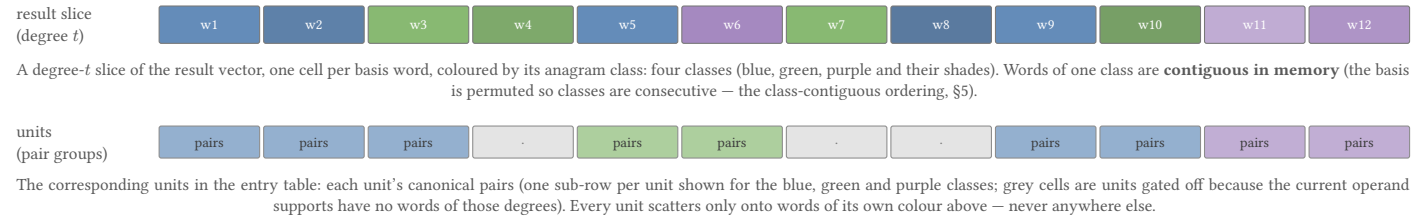
Grouping by content

Group all degree- t basis words by their content vector γ . Each group — an **anagram class** — is the set of words obtainable from one another by permuting letters. Now consider a canonical pair (u, v) with $\gamma(u) + \gamma(v) = \gamma$, $|u| + |v| = t$: by content homogeneity, **everything it produces lands on the single class** (t, γ) .

The kernel groups entries by this pair — a (t, γ) -group is an **anagram unit** — and the payoff is the key structural fact of the whole engine:

Two distinct units never write to the same result word.

Each result word belongs to exactly one anagram class, and each unit feeds exactly one class. Therefore a whole commutation decomposes into units that are **disjoint write regions**: any two of them can be swept concurrently with no locks, no atomics, and no order constraints between them.



Measured shape of this partition (2 letters, degree ≤ 12): 747 basis words, 212 units, ≈ 1700 feasible pairs in one commutation; ≈ 3.5 words and ≈ 8 pairs per unit. At high degree the classes grow quickly (the class at content $(6, 6)$ holds dozens of words), so units are numerous enough to fill many threads — but sized wildly differently, which matters for scheduling (§5).

Why a unit may never be split

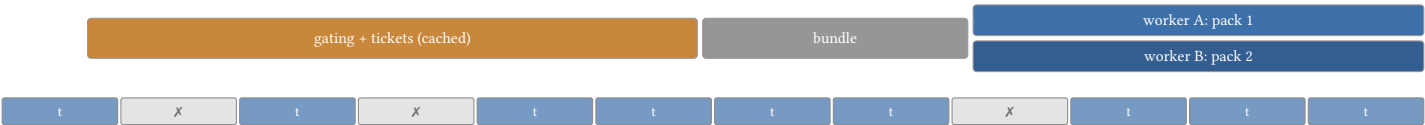
Inside a unit, several canonical pairs feed the **same** result word (the word's decomposition row, §2). The result word accumulates its contributions sequentially — addition of floating-point values is not associative, so **the mathematical definition includes a fixed per-word accumulation order** (the order of the decomposition row). Two workers writing += into the same word would race (lost updates) and would break reproducibility even when they don't (interleaving changes the rounding). Hence:

The anagram unit is the atomic scheduling unit. Packs of parallel work are cut **only** at unit boundaries — a whole unit's entries always stay together, in table order, on one worker. Within a unit the sequential row order is honored by construction; across units no ordering exists at all. This is what makes results **bit-identical to the serial sweep at any thread count** — a property the codebase maintains as an invariant (and guards with cross-thread-count identity tests). The alternative — splitting units and compensating with atomics or privatized partial sums — costs either synchronization on the hot path or extra passes over memory, and an early version of the packer that accidentally cut inside units produced exactly the flaky wrong-results race this boundary exists to prevent.

One commutation, parallelized end to end

A single evaluation of $[X, Y]$ has four phases; two are parallel, two are serial-but-amortized. The architecture's recurring theme is: **anything whose value depends only on the operand supports (which words are nonzero), not on their values, can be precomputed and cached — supports change rarely, values change every fold.**

- | | | |
|------------------|---|---|
| ① Gating | From the two operands' supports, build presence bit-masks, then walk the entry table once to find active segments : the degree-runs whose (p, q) could produce anything. Output: per-unit active entry lists. | serial per call, cached by support fingerprint
part of the cached gating — not recomputed per fold
cheap planning, parallel-safe by §3
the only heavy parallel region |
| ② Tickets | For each active segment, resolve once the per-entry presence tests (is u present in X ? v in Y ? both reversed?) and pack the survivors as compact tickets : the entry's table index plus two orientation bits. | |
| ③ Bundle | Cut the active units into a handful of packs of roughly equal total work, cutting only at unit boundaries. | |
| ④ Sweep | Workers claim packs dynamically. Per ticket: compute the antisymmetric term (1–2 multiplies) and scatter-multiply its decomposition row into the result. | |



Ticket compaction in one segment: surviving entries (blue) become a dense ticket stream; entries that fail presence (grey, X) vanish from the sweep entirely. Measured at 2×12 : 31.7% of all entry visits were dead — tickets eliminate both the dead visits and the repeated bit-tests from every fold of the batch, for the price of one pass per support-change.

Two numbers characterize why these phases are arranged this way:

- **The sweep visits hundreds of thousands of entries per call** (measured $\approx 257,000$ tickets per fold at 2×12); the gating/ticket phases are $O(\text{table})$ once per support change. Across a batch of 1000 folds, the per-fold marginal cost of ① and ② is zero.
- **The serial fraction around a single call** (fork/join, task flattening, bundle build) is a fixed few microseconds — negligible against a millisecond sweep at 2×12 , but **larger than the entire kernel** at 12×2 (66 entries). This asymmetry returns in §7 as the work-adaptive scheduler.

The sweep itself

Per ticket the arithmetic is minimal: one or two multiplies for the term, then one fused multiply-add per decomposition-row element. What surrounds it is memory: the entry record, the operand words it indexes, the decomposed coefficients, and the scattered result words. Measured on real hardware, the per-entry time is **uniform** (11 ns/entry at 1 thread, ≈ 44 cycles) regardless of the pair's degree or row length — which is why balancing packs by entry count balances time (a fancier work metric was implemented, measured slower, and reverted) — and why the sweep parallelizes cleanly until **cache-coherence and streaming bandwidth**, not dependencies, set the limit (per-entry cost rises $\approx 13\%$ merely because two cores stream the same tables; $\approx 9 \times$ at 32 threads in the worst place — the accumulate traffic of §6).

The fold: a DAG of commutations with level barriers

From the BCH series to a DAG

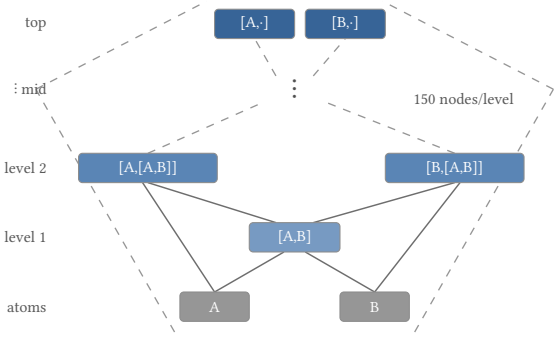
Truncated at degree m , the BCH series $A \oplus B = A + B + \frac{1}{2}[A, B] + \frac{1}{12}[A, [A, B]] - \frac{1}{24}[B, [A, B]] + \dots$ has a fixed bracket structure — it depends only on m , not on data. The fold compiler turns this bracket forest into a DAG:

- Nodes

Bracket subterms, **interned** so shared subtrees (the same nested bracket can appear in many BCH terms) are computed once. Each node is one commutation kernel call — and, by content homogeneity, each node writes **exactly one anagram class** of target words.
- Atoms

The two operands: the running accumulator (A) and the folded-in displacement (B).
- Levels

A node's level is the longest path from the atoms. Nodes of level ℓ read only results of levels $< \ell$, so each level is internally independent — many kernel calls with disjoint result buffers — while level $\ell + 1$ must wait for **all** of level ℓ .



The fold as a DAG — the $[A, B]$ -iterate spine of the truncated BCH structure, shown schematically. Solid edges: consumption of an operand (a node's two operands are results of earlier levels and/or the atoms — note how $[A, B]$ is **shared** by every node above it: **interning** computes it once). The dashed guides suggest the **funnel** shape of the real DAG at working scale (2×12 : 11 levels, 600 nodes, 150 nodes at the widest level): thin at the atoms and the deepest levels, wide in the middle. The levels are the longest path from the atoms; width per level is what a parallel schedule can harvest, and the barrier after each full level is what it must pay.

One parallel dispatch for the whole fold: the pack-slot walk

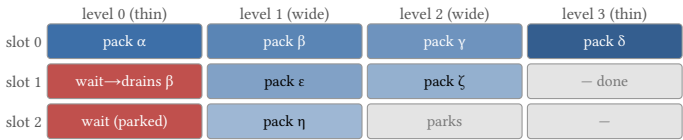
Naively, a fold with L levels pays L separate fork/join dispatches — measured at $\approx 35\%$ of a small fold's time. The folded engine instead plans **all** levels up front (legal because every node's support is fixed for the fold's duration) and dispatches **once**. Dependency order is enforced not by dispatch boundaries but by a small counter protocol:

- Packs per level

Each level's (node, unit) tasks are cut into work-balanced packs — again, only at unit boundaries (§3). Because each node is one anagram class and tasks are node-ordered, every pack is **class-contiguous for free**, which keeps a pack's scatter traffic inside a few cache-resident class blocks.
- Slots

A small number of long-lived worker tasks — never more than workers — each **claims packs off an atomic cursor** per level, executes them, then bumps the level's completion counter.
- Gates

At each level boundary **every** slot waits for the previous level's counter to reach its pack count (acquire/release ordering). A slot that arrives early is not allowed to sit idle-**and**-blocking: before waiting, it **drains any unclaimed packs of the level itself**. That one rule is the whole liveness proof: a level's remaining work is always either claimed-and-running or claimable by the waiter, so the walk cannot deadlock no matter how the pool is oversubscribed.



The walk across one fold (3 slots, 4 levels; shaded = sweep work, red = waiting). Barriers are implicit in the counters; empty levels or low-work phases are where slots park (and free their workers for co-scheduled work) after draining the cursor. Per-word accumulation order is exactly the serial schedule's: packs never split a unit.

The batch: amortizing everything except the serial dependence

A path of 1000 segments is 1000 folds. Running them through the per-fold machinery would re-plan, re-gate, re-gather, and re-allocate 1000 times. The batched driver instead recognizes what is invariant **across** folds and pays it once:

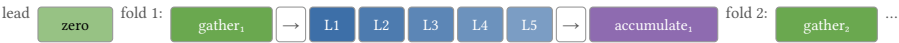
- Support fixed point

After the first few folds the accumulator is dense; every node's "which words does it write" list becomes a fixed point. From then on: no per-fold gating, no list rebuilds — the planner's whole output (gates, tickets, packs) is reused verbatim for the remaining folds. Eligibility for batching is exactly this fixed point plus a shared displacement support.
- Class-space accumulator

The accumulator stays in the class-contiguous basis order across the whole batch (gathered once, written back once at the end), removing two full-basis permutations per fold.
- Compact node buffers

Each node's result lives in a private scratch buffer holding only the degree slices it can touch (degree-blocked, shift-addressed), not the full basis — hundreds of nodes $\times$ full-basis buffers would be megabytes and cache-hostile; degree slices cut that $5 \times$.
- One stage chain, one walk

The whole batch is a single chain of stages — per fold: **gather** the displacement (block-parallel), the L sweep levels, then **accumulate** the terms into the accumulator (block-parallel, simultaneously re-zeroing the node buffers for the next fold) — driven by exactly one pack-slot walk, sharing all cursors and counters.



The batch as one stage chain (5 of L sweep levels shown). Parallel work exists **inside** every stage; between stages, barrier counters; between the last accumulate of fold k and the first sweep of fold $k + 1$, a true data dependency — the accumulator. The chain is long and thin by design: the planning and allocation it replaces cost far more than the barrier hops when a fold's sweep work is large, and far less (§7) when it is small.

The accumulate stage deserves one sentence of its own, because it is the batch's second-largest cost: each fold's terms must be added (with BCH weights) into the accumulator, and every node buffer re-zeroed, per fold. It runs block-parallel over class-position ranges — but measured element volume is large ($\approx 575,000$ slot-touches per fold at 2×12 , $\approx 85\%$ of the sweep's own multiply count) because node buffers store whole degree slices while only support positions are live. Trimming the walk to live slots and fusing read-and-zero into one pass made this —63% smaller; what remains is **coherence-limited**: per-element cost grows $9 \times$ from 4 to 32 threads as the shared accumulator region crosses many cores — the one remaining place where more threads actively hurt inside a stage.

Scheduling to the shape of the work

The one structural number that decides whether parallelism helps is **planned sweep work per fold** — visible exactly, for free, inside the plan. The engine therefore sizes its own parallelism:

	planned entries per fold	chosen slots (32-thread pool)	e2e at 32t vs 1t
12×2 (high-dim, low-degree)	66	1 (serial walk)	2.55 → 1.25 ms ✓ (was 16.4 ms!)
8×3	700	1	28.6 → 7.7 ms ✓
3×8	≈74,000	19	338 → 161 ms ✓
2×12 (low-dim, high-degree)	≈257,000	32	1393 → 549 ms ✓

12×2 @32t

2×12 @32t

barrier/sync 92%

sync

sweep

sweep

...

acc

Same engine, two shapes, before work-adaptivity (illustrative shares, measured): at 12×2 the fold’s 66 entries vanish inside the stage machinery (16t = 6.6× slower than 1t); at 2×12 the sweep dominates and 16–32 threads pay. The scheduler’s rule — **slots ≈ planned work + a fixed quantum, capped by the pool, floored at 1** — makes each shape run in the regime that suits it, at the cost of nothing when the work is large.

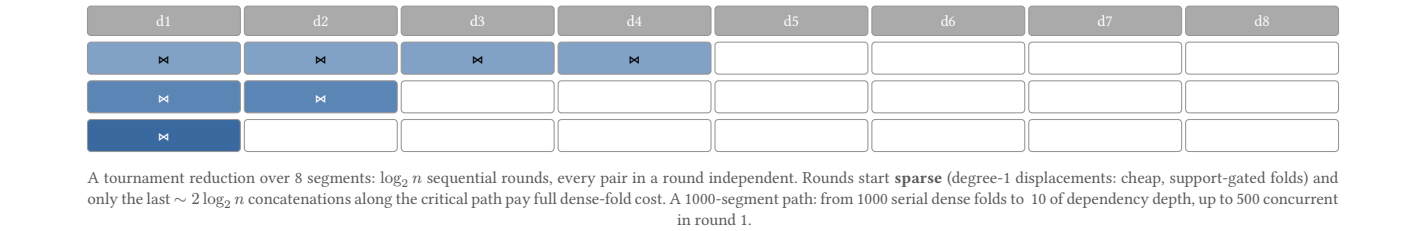
This single rule closed the high-dimension/low-degree pathology: the parallel infrastructure is still there, but a 66-entry fold no longer pays for it.

What threads cannot reach inside a fold — and the associative axis they can

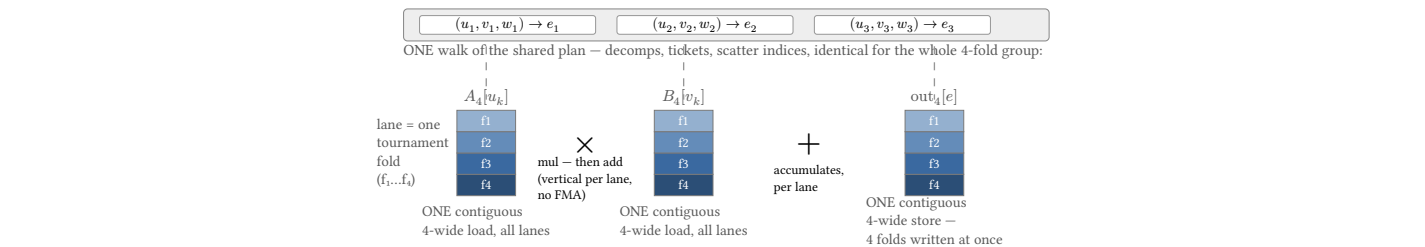
It is worth stating plainly where parallelism **does not** exist, because it is a consequence of the mathematics rather than an implementation gap:

The fold chain is serial along the path. Fold $k + 1$ evaluates its DAG on the output of fold k ; level-0 jobs of fold $k + 1$ read the accumulator written by the accumulate stage of fold k . No reordering of stages, no matter how clever, removes a **data** dependency. Everything shown so far — units, levels, packs, slots, batches — lives strictly inside one concatenation, replicated identically 1000 times down the chain.

The one honest escape hatch already exists in the algebra: BCH concatenation is **associative**. The path’s log signature is an associative reduction of the segment displacements, so the reduction tree may have any shape — including a balanced tournament:



The engine now performs exactly this reduction, adaptively and with no new API surface: when a batch holds at least 16 displacements on a multi-thread pool, the batch driver reduces a **balanced tournament over contiguous chunks** instead of the serial chain. Leaves are not cold per-fold dispatches — each leaf folds its chunk through the same sequential batch engine (warm-up folds, then one steady batched dispatch). Cold per-fold leaves forfeited that amortization and regressed 2×12 by up to 1.27× in measurement; with batched leaves the per-leaf warm-up duplication is the only real tax, and the merge tail — the only remaining serial spine — is $2 \log_2(n/8)$ dense folds deep. The tree shape is a pure function of the displacement count (at most 32 leaves), so results are **bit-stable at any pool size**; exact (rational) coefficients remain bit-identical to the sequential driver, while f64 reassociation shifts rounding only in the last ulps (measured $\approx 10^{-13}$ abs on $\sim 10^2$ magnitudes — accepted, and pinned by tolerance tests against the sequential oracle plus exactness oracles for rationals). The §7 discipline applies at the tree level too: leaves and merges run through the same work-adaptive machinery, so a thin regime’s early rounds do not pay for scheduling. Measured end-to-end (full grid in the regime map below): 12×2 1.84→0.50 ms @8t ($6.7 \times$ at 10000 segments), 8×3 7.94→2.08 ms @32t, 3×8 154→92 ms @32t, and 2×12 — the regime that **needs** the intra-fold machinery most — 525→390 ms @16t ($2.0 \times$ at 10000 segments): 128 batched leaves plus 127 merges replace 1000 dense folds. Single-threaded runs are unaffected: the driver keeps the sequential chain there, so the reformulation is upside-only on every measured grid. The tournament’s folds then compose with a second multiplier, **cohort execution**. A round’s folds all walk the same data-independent plan, so the engine executes four of them as one SIMD walk over lane-interleaved buffers:



Regime map: what was measured

grid	stock, best config	today, best config	gain	why
2×12	1393 ms @32t	390 ms @16t	3.6×	Every intra-fold layer engaged — plus the tournament: 128 batched leaves + 127 merges replace 1000 dense folds ($2.0 \times$ alone at 10000 segments). (<i>low-d, high-m</i>)
3×8	331 ms @8t	92 ms @32t	3.6×	Same; tournament gain grows with pool ($1.7 \times$ @32t).
12×2	2.55 ms @1t (16t: 16.7 ms!)	0.50 ms @8t	5.1×	66 entries/fold: adaptivity runs the fold serial — the tournament puts map and threads come back
8×3	8.02 ms @2t (32t: 28.6 ms)	2.08 ms @32t	3.9×	lanes on the fold AXIS ($6.7 \times$ vs stock at 10000 segments). (<i>high-d, low-m</i>)
				700 entries/fold: formerly on the serial/parallel boundary — same mechanism as 12×2 .

End-to-end log signatures of 1000-segment random paths, criterion medians, 32-core machine. “Stock” = the engine before the parallelism work documented in the companion notes. Single-threaded runs keep the sequential driver (bit-identical there); parallel paths reassociate floating-point folds — exact types unchanged, ulps pinned by oracle tests.

Summary. Parallelism in this engine is **structural**, not opportunistic: the algebra partitions work into anagram units that never share a write; the fold DAG partitions those units into dependency levels; the batch amortizes all support-invariant planning; the scheduler spends threads only where measured work justifies the barriers. The fold chain along the path is a data dependency of the BCH recursion itself — and the tree reduction of §8, now performed adaptively by the batch driver, is the only mechanism mathematics offers for crossing it.